

Módulo 1.2: Librerías Fundamentales para Machine Learning

Curso: Machine Learning con Python (IFCD093PO)

Duración estimada: 12 horas

Objetivos del Módulo

En este módulo, dominarás las tres librerías que forman la base de casi cualquier proyecto de Data Science y Machine Learning en Python. Al finalizar, serás capaz de:

- ✓ **NumPy:** Realizar cálculos numéricos eficientes con arrays multidimensionales.
- ✓ **Pandas:** Cargar, limpiar, manipular y analizar datos tabulares con DataFrames.
- ✓ **Matplotlib y Seaborn:** Crear visualizaciones de datos informativas y estéticamente agradables.

Estas herramientas son el pan de cada día de un científico de datos. ¡Vamos a dominarlas!

Tabla de Contenidos

1. NumPy: Computación Numérica

- ¿Qué es NumPy y por qué es tan rápido?
- Creación de Arrays
- Operaciones y Broadcasting
- Indexación y Slicing
- Estadísticas y Álgebra Lineal

2. Pandas: Manipulación de Datos

- Series y DataFrames
- Lectura y Escritura de Datos (CSV)
- Selección y Filtrado (loc, iloc)
- Agrupación (GroupBy)
- Manejo de Datos Faltantes

3. Visualización: Matplotlib y Seaborn

- Anatomía de un Gráfico con Matplotlib
- Gráficos Estadísticos con Seaborn

4. Resumen y Próximos Pasos

Preparación: Importar las librerías

Por convención, siempre importamos estas librerías con alias específicos. Esto hace el código más corto y legible.

```
In [ ]: # Importar Las Librerías
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Configuraciones recomendadas para una mejor visualización
sns.set_style("whitegrid") # Estilo de Los gráficos de Seaborn
plt.rcParams["figure.figsize"] = (10, 6) # Tamaño por defecto de Los gráficos
```

1. NumPy: Computación Numérica

1.1 ¿Qué es NumPy y por qué es tan rápido?

NumPy (Numerical Python) es la librería fundamental para la computación científica en Python. Su objeto principal es el **array multidimensional** (`ndarray`).

¿Por qué es más rápido que las listas de Python?

- **Memoria Contigua:** Los elementos de un array de NumPy se guardan uno al lado del otro en la memoria, permitiendo un acceso mucho más rápido.
- **Código Optimizado:** Muchas de sus operaciones están escritas en C o Fortran, lenguajes de bajo nivel mucho más rápidos que Python.
- **Vectorización:** Permite aplicar una operación a todo un array a la vez, sin necesidad de bucles `for` explícitos en Python.

```
In [ ]: # Comparación de velocidad: NumPy vs Listas
lista_py = list(range(1_000_000)) # Lista de Python con 1 millón de números
array_np = np.arange(1_000_000) # Array de NumPy con 1 millón de números

print("Calculando el cuadrado de 1 millón de números:")
```

```
# Con Listas de Python (usando un bucle)
print("\nCon listas de Python:")
%timeit [x**2 for x in lista_py] # Usando comprensión de listas

# Con NumPy (vectorizado)
print("\nCon NumPy:")
%timeit array_np*2 # Operación vectorizada
```

1.2 Creación de Arrays

Hay muchas formas de crear arrays en NumPy.

```
In [ ]: # Desde una lista de Python
array_1d = np.array([1, 2, 3, 4, 5])
print("Array 1D:", array_1d)

array_2d = np.array([[1, 2, 3], [4, 5, 6]])
print("\nArray 2D (matriz):\n", array_2d)

# Con funciones de NumPy
array_zeros = np.zeros((2, 3)) # Array de ceros
print("\nArray de ceros:\n", array_zeros)

array_ones = np.ones((3, 2)) # Array de unos
print("\nArray de unos:\n", array_ones)

array_range = np.arange(0, 10, 2) # Similar a range() de Python
print("\nArray con arange(0, 10, 2):", array_range)

array_linspace = np.linspace(0, 1, 5) # 5 números espaciados igualmente entre 0 y 1
print("\nArray con linspace(0, 1, 5):", array_linspace)

array_random = np.random.rand(2, 3) # Números aleatorios entre 0 y 1
print("\nArray aleatorio:\n", array_random)
```

1.3 Operaciones y Broadcasting

Las operaciones se aplican elemento a elemento. **Broadcasting** es la habilidad de NumPy de tratar con arrays de diferentes formas durante las operaciones aritméticas.

```
In [ ]: a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Operaciones elemento a elemento
print("Suma:", a + b)
print("Multiplicación:", a * b)

# Broadcasting: operando un array con un escalar
print("\nBroadcasting:")
print("Array + 10:", a + 10)
print("Array * 2:", a * 2)

# Ejemplo más complejo de broadcasting
matriz = np.array([[1, 2, 3], [4, 5, 6]])
vector = np.array([10, 20, 30])
print("\nMatriz + Vector:")
print(matriz + vector) # El vector se "estira" para sumarse a cada fila
```

1.4 Indexación y Slicing

Funciona de forma similar a las listas, pero con más dimensiones.

```
In [ ]: data = np.arange(1, 13).reshape(3, 4) # Crea un array de 1 a 12 y lo remodela a 3x4
print("Matriz de datos:\n", data)

# Acceder a un elemento: data[fila, columna]
print("\nElemento en la fila 1, columna 2:", data[1, 2]) # Es 7

# Slicing
print("\nPrimera fila completa:", data[0, :])
print("Segunda columna completa:", data[:, 1])
print("\nSubmatriz (primeras 2 filas, columnas 1 y 2):\n", data[0:2, 1:3])

# Indexación booleana (muy potente)
mayores_que_5 = data > 5
print("\nMáscara booleana (elementos > 5):\n", mayores_que_5)
print("\nElementos mayores que 5:", data[mayores_que_5])
```

✓ Ejercicio 1 (NumPy): Normalización de Datos

La normalización Z-score es una técnica muy común en ML. La fórmula es: $(x - \text{media}) / \text{desv_estandar}$.

1. Crea un array de NumPy con 10 números aleatorios (ej: `np.random.randint(50, 100, 10)`).
2. Calcula la media (`.mean()`) y la desviación estándar (`.std()`) del array.
3. Aplica la fórmula de Z-score para normalizar los datos.
4. Imprime el array original y el normalizado.

```
In [ ]: # Tu código aquí
array_sin = np.random.randint(50,100,10)
des_est = array_sin.std()
media = array_sin.mean()
z_score = (array_sin - media) / des_est
print(f"Array original {array_sin}")
print(f"Array normalizado {z_score}")
```

►  Solución

2. Pandas: Manipulación de Datos

Pandas es la librería esencial para trabajar con datos "estructurados" (tablas, como en Excel o SQL). Se construye sobre NumPy.

2.1 Series y DataFrames

- **Serie:** Un array unidimensional etiquetado (como una columna de una tabla). Puede contener cualquier tipo de dato.
- **DataFrame:** Una tabla bidimensional etiquetada (como una hoja de cálculo). Es la estructura de datos principal de Pandas.

```
In [ ]: # Importamos Librería
import pandas as pd
# Creación de una Serie
serie_poblacion = pd.Series([8_000_000, 3_200_000, 1_600_000],
                             index=["Nueva York", "Madrid", "Barcelona"])
print("Serie de población:\n", serie_poblacion)

# Creación de un DataFrame desde un diccionario
datos = {
    'País': ['España', 'Francia', 'Alemania', 'Italia'],
    'Capital': ['Madrid', 'París', 'Berlin', 'Roma'],
    'Población (millones)': [47, 65, 83, 60]
}
df = pd.DataFrame(datos)

print("\nDataFrame de países:")
display(df) # display() es mejor que print() para DataFrames en notebooks
```

2.2 Lectura y Escritura de Datos (CSV)

Una de las tareas más comunes es cargar datos desde un archivo. Pandas lo hace muy fácil, especialmente con archivos CSV (Comma-Separated Values).

```
In [ ]: # Vamos a crear un archivo CSV de ejemplo para poder leerlo
contenido_csv = """ID,Nombre,Edad,Salario
1,Ana,28,50000
2,Luis,34,62000
3,Marta,45,75000
4,Javier,22,45000
5,Sofia,39,
"""
with open("empleados.csv", "w") as f:
    f.write(contenido_csv)

# Leer el archivo CSV en un DataFrame
df_empleados = pd.read_csv("empleados.csv")

print("DataFrame cargado desde 'empleados.csv':")
display(df_empleados)

# Métodos básicos para explorar un DataFrame
print("\nPrimeras 3 filas (.head(3)):" ) # Muestra las primeras 3 filas
display(df_empleados.head(3))

print("\nInformación del DataFrame (.info()):" ) # Tipos de datos y valores nulos
df_empleados.info()

print("\nEstadísticas descriptivas (.describe()):" ) # Estadísticas para columnas numéricas
display(df_empleados.describe())
```

2.3 Selección y Filtrado (loc, iloc)

Hay dos formas principales de seleccionar datos:

- **.loc**: Selecciona por **etiquetas** (nombres de filas/columnas).
- **.iloc**: Selecciona por **posición** entera (índices numéricos).

```
In [ ]: # Seleccionar una columna
nombres = df_empleados['Nombre']
print("Columna 'Nombre':\n", nombres)

# Seleccionar múltiples columnas
nombre_y_salario = df_empleados[['Nombre', 'Salario']]
print("\nColumnas 'Nombre' y 'Salario':")
display(nombre_y_salario)

# .iloc: seleccionar la primera fila (posición 0)
primera_fila = df_empleados.iloc[0]
print("\nPrimera fila (iloc[0]):\n", primera_fila)
```

```
# .loc: seleccionar la fila con índice 2
fila_indice_2 = df_empleados.loc[2]
print("\nFila con índice 2 (loc[2]):\n", fila_indice_2)

# Filtrado booleano: empleados que ganan más de 60000
salarios_altos = df_empleados[df_empleados['Salario'] > 60000]
print("\nEmpleados con salario > 60000:")
display(salarios_altos)
```

2.4 Agrupación (GroupBy)

El `groupby` es una operación extremadamente potente que implica:

1. **Dividir** los datos en grupos basados en algún criterio.
2. **Aplicar** una función a cada grupo de forma independiente.
3. **Combinar** los resultados en una estructura de datos.

```
In [ ]: # Creemos un DataFrame más complejo
data_ventas = {
    'Departamento': ['Deportes', 'Hogar', 'Deportes', 'Tecnología', 'Hogar', 'Tecnología'],
    'Vendedor': ['Ana', 'Luis', 'Ana', 'Marta', 'Luis', 'Marta'],
    'Ventas': [200, 150, 250, 300, 180, 320]
}
df_ventas = pd.DataFrame(data_ventas)
print("DataFrame de ventas:")
display(df_ventas)

# Agrupar por 'Departamento' y calcular la suma de ventas
ventas_por_depto = df_ventas.groupby('Departamento')['Ventas'].sum()
print("\nSuma de ventas por departamento:")
print(ventas_por_depto)

# Agrupar por 'Vendedor' y calcular la media y el total de ventas
stats_vendedor = df_ventas.groupby('Vendedor')['Ventas'].agg(['mean', 'sum'])
print("\nEstadísticas por vendedor:")
display(stats_vendedor)
```

✓ Ejercicio 2 (Pandas): Análisis de Datos de Películas

1. Crea un DataFrame con datos de películas: `Título`, `Director`, `Año`, `Puntuacion` (de 1 a 10).
2. Filtra y muestra solo las películas con una puntuación mayor o igual a 8.
3. Agrupa por `Director` y calcula la puntuación media de sus películas.

```
In [ ]: # Tu código aquí
datos_películas = {
    'Título': ['Pulp Fiction', 'The Dark Knight', 'Forrest Gump', 'The Dark Knight Rises', 'Inception'],
    'Director': ['Tarantino', 'Nolan', 'Zemeckis', 'Nolan', 'Nolan'],
    'Año': [1994, 2008, 1994, 2012, 2010],
    'Puntuacion': [8.9, 9.0, 8.8, 8.4, 8.8]
}
df_películas = pd.DataFrame(datos_películas)
display(df_películas)
# 1. Filtra las mejores películas
df_buenas = df_películas[df_películas["Puntuacion"] > 8.5]
display(df_buenas)
# 2. Agrupa por director
directores = df_películas.groupby("Director")["Título"].count()
display(directores)
```

► 💡 Solución



3. Visualización: Matplotlib y Seaborn

Una imagen vale más que mil palabras (o mil filas de datos). La visualización es clave para entender patrones, tendencias y outliers (valores atípicos). °

- **Matplotlib:** Es la librería base. Potente y altamente personalizable, pero a veces verbosa.
- **Seaborn:** Construida sobre Matplotlib. Ofrece una interfaz de más alto nivel para crear gráficos estadísticos atractivos con menos código.

3.1 Anatomía de un Gráfico con Matplotlib

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
import seaborn as sb

In [ ]: # Datos de ejemplo
x = np.linspace(0, 10, 100)
y_seno = np.sin(x)
y_coseno = np.cos(x)

# Crear el gráfico
plt.plot(x, y_seno, label='Seno(x)', color='blue', linestyle='--')
plt.plot(x, y_coseno, label='Coseno(x)', color='red', linestyle='--')

# Añadir títulos y etiquetas
plt.title('Funciones Seno y Coseno', fontsize=16, fontweight='bold')
```

```
plt.xlabel('Eje X', fontsize=12)
plt.ylabel('Eje Y', fontsize=12)

# Añadir Leyenda
plt.legend()

# Mostrar el gráfico
plt.show()
```

3.2 Gráficos Estadísticos con Seaborn

Seaborn simplifica la creación de gráficos comunes en el análisis de datos.

```
In [ ]: # Usaremos el DataFrame de empleados que creamos antes
print("Recordatorio del DataFrame de empleados:")
display(df_empleados)

# Gráfico de dispersión (scatterplot) para ver la relación entre edad y salario
plt.figure()
sns.scatterplot(data=df_empleados, x='Edad', y='Salario', hue='Nombre', s=200)
plt.title('Relación entre Edad y Salario', fontsize=16)
plt.show()

# Gráfico de barras (barplot) para comparar salarios
plt.figure()
sns.barplot(data=df_empleados, x='Nombre', y='Salario')
plt.title('Salario por Empleado', fontsize=16)
plt.show()

# Histograma para ver la distribución de una variable
plt.figure()
sns.histplot(data=df_empleados, x='Salario', kde=True) # kde=True añade una curva de densidad
plt.title('Distribución de Salarios', fontsize=16)
plt.show()
```

✓ Ejercicio 3 (Visualización): Gráfico de Películas

Usando el DataFrame de películas del ejercicio anterior:

1. Crea un **gráfico de barras** que muestre la puntuación de cada película.
2. Crea un **gráfico de conteo** (countplot) que muestre cuántas películas ha dirigido cada director en el dataset.

```
In [ ]: # Tu código aquí
# 1. Gráfico de barras de puntuaciones
plt.figure()
sns.barplot(data=df_peliculas, x='Titulo', y='Puntuacion')
plt.title('Puntuación de Películas', fontsize=16)
plt.show()
# 2. Gráfico de conteo de directores
plt.figure()
sns.countplot(data=df_peliculas, x="Director")
plt.title('Películas por Directo', fontsize=16)
plt.show()
```

► 💡 **Solución**

4. Resumen y Próximos Pasos

¡Felicidades! Has dominado las librerías fundamentales

✓ Lo que has aprendido:

1. NumPy

- Crear y manipular arrays multidimensionales.
- Realizar operaciones matemáticas vectorizadas y usar broadcasting.
- Seleccionar datos con slicing e indexación booleana.

2. Pandas

- Trabajar con Series y DataFrames.
- Cargar datos desde archivos CSV y explorarlos.
- Filtrar y seleccionar datos de manera precisa con `.loc`, `.iloc` y condiciones booleanas.
- Agrupar datos y calcular estadísticas con `groupby`.

3. Matplotlib y Seaborn

- Entender la estructura básica de un gráfico.
- Crear gráficos de líneas, barras, dispersión e histogramas.
- Usar Seaborn para crear visualizaciones estadísticas atractivas con poco código.

Próximo Módulo: Introducción al Machine Learning

Ahora que sabes cómo manejar y visualizar datos, estás listo para el siguiente gran paso: **¡entrenar tu primer modelo de Machine Learning!**

En el próximo módulo aprenderás:

- ¿Qué es realmente el Machine Learning?
- Los diferentes tipos de aprendizaje (supervisado, no supervisado).
- El flujo de trabajo completo de un proyecto de ML, desde la carga de datos hasta la evaluación del modelo.
- Conceptos clave como *overfitting*, *underfitting* y *validación cruzada*.

¡Aquí es donde todo lo que has aprendido se une para crear algo increíble!